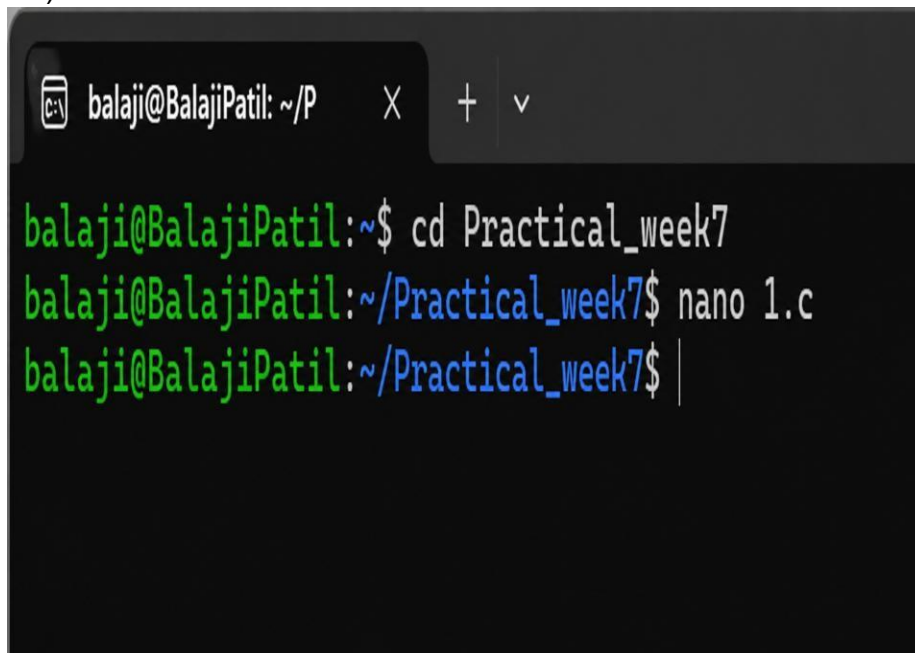


PRACTICAL WEEK 7

Task 1: Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

1) Create a file

A terminal window with a dark background. The title bar shows 'balaji@BalajiPatil: ~/P' with window control icons. The terminal content shows three lines of commands: 'balaji@BalajiPatil:~\$ cd Practical_week7', 'balaji@BalajiPatil:~/Practical_week7\$ nano 1.c', and 'balaji@BalajiPatil:~/Practical_week7\$ |' with a cursor at the end.

```
balaji@BalajiPatil:~$ cd Practical_week7
balaji@BalajiPatil:~/Practical_week7$ nano 1.c
balaji@BalajiPatil:~/Practical_week7$ |
```

Then press enter to open command

2) write a code

```
balaji@BalajiPatil: ~/P  × + ∨
GNU nano 8.7.1 1.c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;           // Global variable
int global_uninit;            // Global uninitialized variable

void code_function()
{
    printf("Address of code function : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20; // Static variable
    int stack_var = 30;         // Stack variable

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Linux Process Memory Address Layout\n\n");

    code_function();

    printf("Address of global variable      : %p\n", (void *)&global_var);
    printf("Address of global uninitialized : %p\n", (void *)&global_uninit);
    printf("Address of static variable      : %p\n", (void *)&static_var);
    printf("Address of heap variable        : %p\n", (void *)heap_var);
    printf("Address of stack variable       : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

3)

- 1) To save press control O
- 2) To come back to shell press control X
- 3) Then DO compilation

```
balaji@BalajiPatil: ~/P  × + ∨
balaji@BalajiPatil:~$ cd Practical_week7
balaji@BalajiPatil:~/Practical_week7$ nano 1.c
balaji@BalajiPatil:~/Practical_week7$ gcc 1.c -o 1
balaji@BalajiPatil:~/Practical_week7$ |
```

OUTPUT:

```
pranay:~/P
pranay:~$ cd Practical_week7
pranay:~/Practical_week7$ nano 1.c
pranay:~/Practical_week7$ gcc 1.c -o 1
pranay:~/Practical_week7$ ./1
Linux Process Memory Address Layout

Address of code function      : 0x6056213561c9
Address of global variable   : 0x605621359010
Address of global uninitialized : 0x60562135901c
Address of static variable    : 0x605621359014
Address of heap variable      : 0x6056588e1010
Address of stack variable     : 0x7fff5cef318c
pranay:~/Practical_week7$ |
```

Task-2: Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```

pranay:~/P
GNU nano 8.7.1 2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
int global_uninit;

int main()
{
    static int static_var = 20;
    int stack_var = 30;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Process ID: %d\n", getpid());
    printf("Code : %p\n", (void *)main);
    printf("Global : %p\n", (void *)&global_var);
    printf("Static : %p\n", (void *)&static_var);
    printf("Heap : %p\n", (void *)&heap_var);
    printf("Stack : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}

```

OUTPUT:

```

pranay:~/P
GNU nano 8.7.1      2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
int global_uninit;

int main()
{
    static int static_var = 20;
    int stack_var = 30;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Process ID: %d\n", getpid());
    printf("Code : %p\n", (void *)main);
    printf("Global : %p\n", (void *)&global_var);
    printf("Static : %p\n", (void *)&static_var);
    printf("Heap : %p\n", (void *)&heap_var);
    printf("Stack : %p\n", (void *)&stack_var);

    printf("\nProcess is running....\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}

```

```

pranay:~/P
pranay:~/Practical_week7$ nano 2.c
pranay:~/Practical_week7$ gcc 2.c -o 2
pranay:~/Practical_week7$ ./2
Process ID: 845
Code : 0x5c8d5ba951c9
Global : 0x5c8d5ba98010
Static : 0x5c8d5ba98014
Heap : 0x5c8d5ee44010
Stack : 0x7fffebc52aac

Process is running...

```

```

pranay:~/P
pranay:~$ cd Practical_week7
pranay:~/Practical_week7$ nano 1.c
pranay:~/Practical_week7$ gcc 1.c -o 1
pranay:~/Practical_week7$ ./1
Linux Process Memory Address Layout

Address of code function      : 0x6056213561c9
Address of global variable    : 0x605621359010
Address of global uninitialized : 0x60562135901c
Address of static variable    : 0x605621359014
Address of heap variable      : 0x6056588e1010
Address of stack variable     : 0x7fff5cef318c
pranay:~/Practical_week7$ |

```